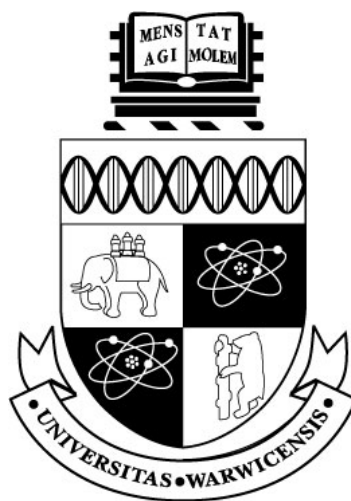




GODOT 4.6 大型可视化行为树的可读性 与运行时检查优化

by

[作者姓名]

A thesis submitted in partial fulfilment
of the requirements for the degree of

游戏工程理学硕士

UNIVERSITY OF WARWICK

华威大学 WMG

2026 年 9 月

目录

目录	ii
插图	v
表格	vi
致谢	vii
声明	viii
摘要	ix
1 引言	1
1.1 研究背景	1
1.2 问题定义与研究空白	1
1.3 研究目标与问题	2
1.4 研究目标与成功标准	2
1.5 主要贡献	3
1.6 论文结构	3
2 文献综述	4
2.1 作为可执行层级的行为树	4
2.2 节点-连线树与扩展性	4
2.3 焦点、上下文与概览	5
2.4 复杂度管理与心理地图	5
2.5 运行时解释与失败定位	6
2.6 研究空白与设计启示	6
3 研究方法	7
3.1 研究策略	7
3.2 实验因素与数据集	7
3.3 正确性与集成评估	8
3.4 视觉验证	8

3.5	人体实验设计	8
3.6	有效性与可复现性	9
4	系统设计与实现	10
4.1	架构与项目边界	10
4.2	资源模型与校验	10
4.3	运行时语义	11
4.4	编辑器编写工作流	11
4.5	Live Debug 与黑板检查	12
4.6	大型树显示机制	13
4.6.1	信息密度控制	13
4.6.2	焦点与结构裁剪	13
4.6.3	导航与概览	13
4.6.4	连线与运行表示	13
4.7	运行时拓扑缓存	14
4.8	演示游戏	14
5	结果与分析	15
5.1	功能正确性	15
5.2	显示密度与上下文结果	15
5.3	路径与概览结果	16
5.4	视觉回归发现	17
5.5	运行时缓存结果	17
5.6	对应研究问题的结果	18
6	讨论	19
6.1	完整系统作为研究工具	19
6.2	显示结果的解释	19
6.3	Live Debug 作为可读性机制	20
6.4	缓存权衡	20
6.5	可靠性与发布	20
6.6	有效性威胁	21
6.6.1	构念有效性	21
6.6.2	内部有效性	21
6.6.3	外部有效性	21
6.6.4	研究者与参与者有效性	21
6.7	实践启示	21
7	结论	22
7.1	总结	22

7.2 贡献	22
7.3 局限	22
7.4 未来工作	23
7.5 最终结论	23
参考文献	24
A 功能参考	26
B 复现与证据索引	27
C 计划中的参与者实验	28
D 初稿完成清单	29

插图

4.1	Godot 编辑器、行为树资源、Runner 与 Actor 之间的插件架构和数据流。	10
4.2	仅位于编辑器中的活动路径和失败原因显示。	12
4.3	同一行为树的完整细节基线和紧凑显示。	13
5.1	364 节点树上主要方法的几何和显著性效果。	16
5.2	焦点 + 上下文、结构裁剪和显著性导航的实现示例。	17
5.3	开启与关闭安全拓扑缓存时的 Tick 扩展。	18

表格

4.1	已实现的运行时节点语义。	11
5.1	最终核心自动回归。	15
5.2	固定生成树的主要显示测量。	16
5.3	活动分支淡化与增强小地图。	17
5.4	按树规模和 Runner 数统计的运行时中位值。	18
A.1	最终编辑器的独立显示功能。	26

致谢

此处预留给作者感谢导师、未来用户实验的参与者，以及帮助测试软件和审阅论文的人。提交前应由作者自行完成最终文字。

声明

本论文作为本人申请游戏工程理学硕士学位的材料提交给华威大学。论文由本人完成，且未曾用于申请其他学位。除明确引用的资料外，本文所述软件、测试数据和分析均来自本项目。本文没有报告任何参与者实验结果：人体对比实验协议已经准备，但招募和数据采集尚未进行。正式提交前，作者必须根据学校规定审阅并个性化本声明，包括如实说明生成式人工智能的使用方式。

摘要

可视化行为树常用于编写游戏智能体的决策，但随着节点数量以及运行时信息增加，传统节点-连线编辑器会变得难以检查。本文设计并评估了一个 Godot 4.6 编辑器插件，将完整、可执行的行为树系统与可独立开关的大树显示和实时调试技术结合。运行时支持有序、响应式、随机、并行、重复和定时控制流，支持 Actor 方法调用、类型化黑板、Decorator 以及仅存在于编辑器中的运行检查。编辑器实现紧凑卡片、语义缩放、鱼眼焦点 + 上下文、子树折叠与聚焦、搜索高亮、概览 + 细节、稳定布局、多种连线、当前路径强调和失败原因标注。

评估使用确定性的 31、121 和 364 节点树、复杂二维守卫场景、自动语义与 GUI 测试以及真实 GPU 视觉回归。核心 447 条断言全部通过，此外还有人体实验固定场景视觉测试 21/21、运行时性能与缓存正确性 511/511、洁净发布包检查 53/53。在 364 节点树上，紧凑卡片将卡片总面积降低 55.9%，子树聚焦将可见节点降低 97.5%，折叠降低 99.5%，搜索淡化 99.7% 的非目标节点，小地图保持 100% 节点覆盖。安全拓扑缓存使 31 节点树的 Tick 中位成本降低 15.98–26.46%，121 节点降低 52.61–56.02%，364 节点降低 76.65–78.00%。这些结果证明了实现正确性和几何显示效果，但不能单独证明人的可读性得到改善，因此本文把对照用户实验作为最主要的工作。

1 引言

1.1 研究背景

现代游戏智能体需要在巡逻、追击、攻击、撤退和恢复等行为之间做出选择。对于小型原型，把全部决策写在单一过程式脚本中是可行的；但随着状态和中断条件增加，控制流会越来越难以检查和修改。行为树（Behaviour Tree, BT）把决策逻辑表示为控制节点和叶节点组成的层级结构。复合节点决定如何选择子节点，条件和动作则把抽象决策结构连接到游戏状态与 Actor 代码。高层策略可以独立于移动、动画或战斗方法进行编辑，这是行为树在游戏开发中的主要价值之一 (Colledanchise and Ögren, 2018)。

商业引擎已经证明可视化行为编写的实用性。Unreal Engine 将节点-连线树、Decorator、黑板和编辑器运行状态结合 (Epic Games, 2026)。Godot 4.6 提供 EditorPlugin、GraphEdit 和 GraphNode 等通用扩展接口，但并没有规定完整的可视化行为树工作流 (Godot Engine contributors, 2026b,a)。因此本项目同时面对工程问题 and 研究问题：插件首先必须能正确执行行为树并接入任意 Actor；在此基础上，随着节点和运行标注增加，图形表示还必须保持可检查性。

传统节点-连线树能够直接表达父子拓扑，但会随宽度和深度增长而迅速占用空间。行为树卡片还可能显示参数、描述、Decorator、黑板键和执行状态，比只显示标签的普通层级图更大。当数十或数百张卡片放入有限画布时，开发者必须频繁缩放、平移和搜索，当前执行路径也可能淹没在非活动分支中。这不只是美观问题，因为节点图正是设计者定位任务、理解顺序、修改条件和诊断失败的主要工具。

因此，本项目把一个功能完整的 Godot 行为树插件作为大型树显示研究的平台。在保持上下端口和“同级从左到右执行”的语义基础上，加入可独立开关的焦点 + 上下文、复杂度管理、导航和运行时解释技术。每项技术都能单独关闭，一方面防止视觉功能故障拖垮编辑器，另一方面保留可复现的基线比较条件。

1.2 问题定义与研究空白

树和图可视化领域已经提出许多方法。鱼眼和焦点 + 上下文视图会为焦点附近的元素分配更多空间 (Furnas, 1986; Lamping et al., 1995)；概览 + 细节保留全

局参考视图 (Cockburn et al., 2008); 多列布局可以改善大扇出树的浏览 (Song et al., 2010); 增量展开和折叠可以降低图复杂度, 同时尽量保持用户的心理地图 (Dogrusoz et al., 2018; Misue et al., 1995); 大型分层图研究还表明, 路径任务的效率取决于具体表示和任务 (Bae and Watson, 2011)。

然而, 这些研究没有直接回答游戏行为树编辑器应如何组合上述机制。行为树与文件目录等普通层级不同: 同级顺序具有执行语义; 叶节点会调用代码; Decorator 可能阻止分支运行; 图的状态会在每个运行 Tick 中变化。一个可用的方案必须保持执行顺序, 支持编辑与撤销, 公开黑板状态, 并在不向游戏画面加入调试覆盖层的前提下解释失败。现有研究可以指导单项设计, 但仍需要一个集成、可执行且可复现评估的 Godot 实现。

1.3 研究目标与问题

本文目标是设计、实现并评估一个 Godot 4.6 可视化行为树插件, 使其既能驱动真实 NPC, 又能为大型树提供稳健、可切换的检查机制。

研究问题如下:

RQ1: Godot 编辑器插件能否提供控制非简单二维游戏智能体所需的核心编写、运行时、黑板、Decorator 和实时调试能力?

RQ2: 可独立开关的显示技术会如何影响 31、121 和 364 节点行为树的可测属性, 包括可见节点比例、卡片面积、信息字段、目标淡化、概览覆盖率和更新成本?

RQ3: 在不改变执行结果、并能安全处理资源原地修改的前提下, 拓扑缓存能否降低不同树规模和 Actor 数量下的 Tick 成本?

RQ2 有意讨论可测显示属性, 而不直接宣称可用性得到改善。是否降低任务时间、错误率或认知负荷必须由参与者数据验证。本文已经设计相应实验, 但不会伪造参与者结果。

1.4 研究目标与成功标准

项目的具体目标包括: 实现可序列化的树、节点、Decorator 和类型化黑板资源; 实现 Root、Sequence、Selector、Reactive Selector、Random Selector、Parallel、Repeat、Wait、Action 和 Condition 的 SUCCESS、FAILURE 与 RUNNING 语义; 允许可复用组件为 NPC 指定树和 Actor, 并由叶节点调用 Actor 方法; 提供创建、连接、断开、拖拽、类型化编辑、校验、保存、加载、撤销和重做; 只在 Godot 编辑器中显示路径、节点状态、黑板和失败原因; 为大型树显示方法提供独立持久化开关和安全复位路径; 构造确定性大树与复杂二维场景; 最后以可复现证据评价正确性、几何显示、视觉回归、性能和洁净安装。

成功标准不仅是插件能够启动。每类运行节点需要聚焦语义测试，编辑操作必须通过保存/重载和历史测试，游戏场景必须展示索敌、追击、攻击、搜索、撤退与恢复，视觉功能必须在真实渲染器上检查，缓存与非缓存执行必须得到相同终态。即使断言计数通过，只要日志出现非零退出、脚本错误、泄漏、原生崩溃或非法内存访问，仍判定测试失败。

1.5 主要贡献

本文的主要贡献为：

- 一个可分发的 Godot 4.6 行为树插件，包含可执行复合节点、叶节点、Decorator、Actor 方法集成和类型化黑板；
- 一个仅位于编辑器中的实时调试桥，可报告活动路径、黑板值和失败原因，而不把调试 UI 耦合到游戏；
- 一组集成的 19 项可独立开关显示功能，覆盖紧凑/语义表示、焦点 + 上下文、子树裁剪、导航、路径强调、无障碍编码和连线显示；
- 一套可复现评估材料，包括 31、121、364 节点资源、GPU 截图、原始 CSV、公式工作簿和参与者实验协议；
- 一种精确拓扑签名缓存，可检测相同节点数量下的换序、改父级、Decorator 改归属和资源实例替换，并包含九组规模/负载测量。

1.6 论文结构

第2章批判性回顾行为树和可扩展层级可视化；第3章定义设计科学与实验方法；第4章说明资源、运行时、编辑器、显示机制、游戏场景和测试架构；第5章报告功能、几何、视觉与运行时结果；第6章结合研究问题和文献讨论结果与局限；第7章总结贡献和未来工作。附录提供功能、证据与复现实验索引。

2.1 作为可执行层级的行为树

行为树会重复评估一棵有根层级，并传播 `SUCCESS`、`FAILURE` 或 `RUNNING` 三种状态。`Sequence` 通常在第一个失败或运行中的子节点停止；`Selector` 在第一个成功或运行中的子节点停止；叶节点读取状态或执行动作。这个小型状态接口使复杂逻辑能够组合，并能明确表达中断策略 (Colledanchise and Ögren, 2018)。

三状态接口看似简单，却包含关键实现选择。运行中的 `Sequence` 可以记住当前索引，而 `Reactive Selector` 需要重新检查高优先级条件并抢占较低优先级分支；`Random Selector` 在子节点运行期间应保持同一次选择；`Parallel` 需要成功和失败阈值；`Repeat` 与 `Wait` 需要在重启或中断时清除记忆。因此，编辑器显示和运行语义不能完全分离：子节点顺序、Decorator 归属和节点身份都会影响执行，而且必须正确序列化。

黑板在感知、条件和动作之间提供共享键值状态。Unreal Engine 的工作流说明了把黑板、Decorator 和运行观察结合的价值 (Epic Games, 2026)。但本项目没有必要重现商业系统的全部能力。合理基线是：系统能够表达有意义的决策、调用项目 Actor 方法、验证黑板键，并解释分支为何运行或被拒绝。

2.2 节点-连线树与扩展性

节点-连线图适合行为树，因为能直接表示祖先关系和同级分组。经典整洁树算法追求层级一致、子树不重叠且相对紧凑的布局 (Reingold and Tilford, 1981)。`SpaceTree` 又把节点-连线结构与动态布局、缩放和选择性展开结合，其评估说明表示和交互会共同影响拓扑理解与重访任务 (Plaisant et al., 2002)。

问题在于规模。卡片占据面积，随着宽度和深度增加，显式连线也会变密。Song 等比较传统、列表和多列视图，发现多列界面在其大扇出层级的浏览和理解任务中更快，总体偏好也更高 (Song et al., 2010)。这一结果适用于包含大量备选行为的 `Selector` 或 `Sequence`，但不能直接照搬：论文中的子节点按字母排序，而行为树的横向顺序可能决定优先级。因此多列行为树必须明确显示顺序，且不能在布局时静默修改资源。

Bae 与 Watson 为大型分层图提出 Quilts 表示 (Bae and Watson, 2011)。其价值不一定是要求行为树完全改用矩阵，而是证明路径任务可能更适合另一种补

充表示。本项目因此保留可编辑节点图，同时加入紧凑文字路径和强活动路径编码，以针对运行时路径定位任务。

2.3 焦点、上下文与概览

Furnas 用兴趣度函数形式化了广义鱼眼视图：综合元素先验重要性和与焦点的距离，决定显示显著程度 (Furnas, 1986)。Lamping 等使用双曲几何在有限区域显示大型层级，对焦点进行放大并压缩远处上下文 (Lamping et al., 1995)。行为树编辑器可以借此读取鼠标所在节点，同时保留周边结构。

但是焦点 + 上下文并不天然优于其他方法。畸变会移动目标并增加点击难度，缩放变化也可能破坏心理地图。Cockburn 等区分概览 + 细节、缩放和焦点 + 上下文，并指出效果取决于任务、实现和切换成本 (Cockburn et al., 2008)。因此本项目把鱼眼最大倍率限制为 1.20，改变卡片内部表示而不是施加不稳定的整体图变换，以节点中心补偿位置，并在关闭时恢复全部几何。同时提供独立固定小地图，使两类技术可以比较或组合。

语义缩放与几何缩放不同。几何缩放改变物理大小，语义缩放改变显示属性。低细节卡片保留类型颜色、图标和短标题；高细节卡片展示方法、参数、描述和 Decorator。它以信息完整性换取密度，却不改变执行图几何。此前插件曾把滚轮与临时节点缩放耦合，造成滚动时节点先缩小再放大；分离信息可见性和图几何后，该问题被消除。

2.4 复杂度管理与心理地图

当不需要同时查看全部节点时，展开-折叠和隐藏-显示可以降低复杂度。Dogrusoz 等提出大型网络的增量复杂度管理方法，包括展开、折叠和布局更新 (Dogrusoz et al., 2018)。该研究与心理地图原则一致：如果一次编辑引起大量无关节点移动，用户就必须重新学习位置 (Misue et al., 1995)。行为树折叠应保留折叠根，报告隐藏内容，并尽量不移动无关分支。

子树折叠和子树聚焦解决不同问题。折叠用摘要替代后代，同时保留树的其他部分；聚焦只显示目标分支和必要上下文。二者都可能隐藏运行节点，因此插件提供隐藏节点数量、两层摘要、路径指示以及显式展开/聚焦控制，而不是无提示地删除视图信息。

搜索高亮与非活动分支淡化属于较柔和的复杂度管理：几何保持不变，只降低非相关内容显著性。这符合 Shneiderman 的“先概览、再缩放和过滤、按需细节”原则 (Shneiderman, 1996)。形状/图标和色盲友好配色则提供冗余通道，避免只用颜色表达节点类型。

2.5 运行时解释与失败定位

实时调试为静态层级增加时间维度。只高亮叶节点不能解释其到达路径；高亮所有历史节点又会混淆当前状态。本项目记录有序的 Root 到叶节点链条、逐节点状态和当前失败解释。非活动分支可以淡化，独立路径摘要在图缩小时仍可阅读。

Decorator 让失败解释更加重要。Action 本身可能正确，却因为黑板比较、冷却或时间限制而从未执行。在所属节点上显示 Decorator 摘要并附加失败原因，可以把运行证据连接回编写表示；实时黑板进一步显示 Key、运行时类型、值和 Schema 状态。这体现了分层图研究的任务导向原则：调试时用户往往在问“为什么这个分支没有运行”，而不是“完整拓扑是什么”。

2.6 研究空白与设计启示

文献支持多个单项机制，却没有规定如何把它们集成到一个可执行 Godot 行为树编辑器中。本文据此采用四项原则：保留节点-连线拓扑和明确同级顺序作为基线；分离几何放大、语义细节和结构过滤；在折叠与布局中维护空间上下文，并在隐藏细节时提供概览或路径表示；最后，把几何效果和软件正确性与人的任务表现分开评估。最后一项尤其重要：面积或可见节点减少只能证明表示更紧凑，不能证明更容易理解。

3 研究方法

3.1 研究策略

本研究采用设计科学与软件工程结合的策略。研究产物是 Godot 4.6 插件和可执行演示；研究知识来自把行为树语义和可视化文献中的设计要求落实为系统，再测量其正确性、显示属性和运行成本。迭代开发是必要的，因为编辑器集成暴露了单独算法中看不到的问题，例如坐标空间错误、残留 Transform、重复连线和半写入调试快照。

过程分为五个阶段：首先把导师会议要求和论文规范转化为验收门槛；其次建立最小可执行行为树核心和游戏集成；然后加入编辑操作和 Live Debug；随后将显示技术放在独立开关后，每次增加功能都测试关闭与复位；最后用确定性工作负载和分层测试比较完成系统与自身基线。后续任务完成后重新运行完整回归，以防图重建或资源复制改动破坏早期功能。

3.2 实验因素与数据集

三棵确定性生成树分别包含 31、121 和 364 个节点，代表易于编辑的小树、超过导师所提约 50 节点可读性边界的中型树，以及大型压力场景。三棵树使用相同生成规则，使规模成为主要差异；固定目标节点则保证搜索和实时路径截图可重复。

显示实验以完整细节节点-连线图为基线，主要条件为 Compact、Semantic Zoom、Search、Subtree Focus、Subtree Collapse、Fisheye 和 Enhanced Minimap。其余功能通过 19 项开关循环覆盖。指标定义如下：可见比例为可见节点数除以总数；卡片面积为可见卡片最小宽高乘积之和；信息字段表示当前语义层保留的代表字段；淡化节点数表示搜索或路径聚焦降低显著性的节点；鱼眼增益为目标卡片内部倍率；概览覆盖率为小地图表示节点数除以总数；操作时间为当前机器上的重建、更新或开关周期时间。

这些是界面仪器指标，不是心理指标。计划中的人体实验会另外记录任务完成时间、误选、缩放和平移次数、成功率，以及七点评分的可读性与认知负荷。

运行时实验交叉三种树规模、1/10/50 个 Runner 和缓存开/关。每个组合预热后测量七次。测试调整 Tick 数，使各条件执行一个有界且大致可比的总工

作负载。使用 Tick 均值的中位数，以降低临时调度噪声影响；缓存和非缓存还必须返回相同终态。

3.3 正确性与集成评估

测试分为五个核心层。资源/运行时测试检查结构、节点语义、Decorator、Actor、Schema、调试快照和缓存失效；编辑器 GUI 测试实例化插件并检查创建、拖拽、连接、类型字段、保存重载、撤销重做和开关；基础游戏测试检查玩家与敌人交互；复杂竞技场测试覆盖索敌、追击、方向攻击、丢失目标、撤退与治疗；真实 GPU 回归在 1600×900 分辨率捕获 Godot Dummy 渲染器无法提供的可视属性。

判定策略严格：非零退出码、FAIL:、ERROR:、SCRIPT ERROR:、泄漏、资源未释放、非法访问或原生崩溃都会使套件失败。警告则按语义判断：故意调用不存在 Actor 方法是安全测试，应返回 FAILURE；-quit-after 后的文件扫描中断属于预期关闭行为。

3.4 视觉验证

自动断言不能发现所有显示故障。因此视觉协议在 NVIDIA GeForce RTX 5070 Laptop GPU 和 Godot OpenGL 3.3 Compatibility 渲染器上生成稳定截图。核心截图覆盖基线、类型编码、灰度、无障碍配色、单线、紧凑、鱼眼开关、折叠、搜索、运行失败、实时黑板、Schema 编辑、Key 选择、正交连线、捆绑连线和复杂树；另外四张固定截图覆盖 121/364 节点概览、远距搜索目标和确定性 Live Debug 路径。

人工检查关注卡片重叠、端口位置、重复边、文字裁切、残留缩放、视口居中、活动路径连续性、淡化复位和小地图覆盖。正是该方法发现鱼眼虽然状态变量正确，却存在偏右中心和滚轮缩放干扰。

3.5 人体实验设计

计划采用被试内实验，对比 Baseline、Compact、Collapse、Fisheye、Search 和 Minimap。任务包括定位指定 Action、解释固定七节点运行路径、编辑指定 Decorator。条件使用平衡顺序，减少学习效应。工作簿支持 15 人、每人 18 次，共 270 行试验数据。

建议招募 8–15 名具备基础游戏或节点编辑经验的参与者，先完成统一训练。除匿名编号和可选经验类别外，不采集身份信息。正式招募前必须遵守学校伦理和导师要求。由于尚未实施，本文没有推断性人体实验结果。

3.6 有效性与可复现性

固定资源、确定性目标、独立开关、重复样本和集成后全回归增强了内部有效性。构念有效性仍受限制，因为面积和字段数是信息密度代理，不是可读性本身。外部有效性限于一台 Windows 笔记本、Godot 4.6、生成树和当前二维游戏。因此所有时间值都与硬件和工作负载一起报告，不视为跨设备常数。

复现材料包括原始 CSV、截图、测试脚本、公式工作簿、洁净安装打包脚本和 MIT 许可证插件包。人体实验材料与自动证据分开保存，防止未采集的数据被误认为结果。

系统设计4与实现

4.1 架构与项目边界

系统分为编辑器层、资源层、运行时层和验证层（图4.1）。活动演示项目与内容同步的分发插件模板相互分离。游戏场景只依赖序列化资源和运行组件；编辑器控件与 Live Debug 表示不进入游戏 GUI。这一边界避免导出后的智能体行为隐式依赖编写工具。

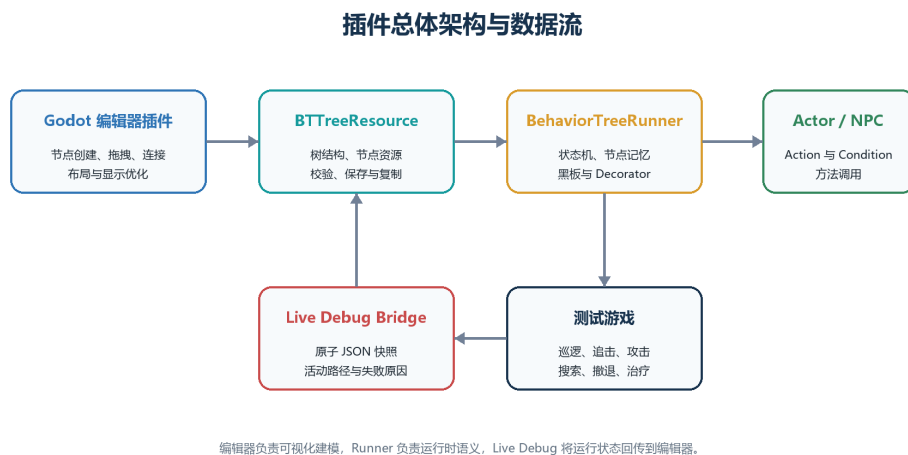


图 4.1: Godot 编辑器、行为树资源、Runner 与 Actor 之间的插件架构和数据流。

BTreeResource 保存 RootID、节点数组和可选黑板 Schema; BTreeNodeResource 保存类型、标题、描述、参数、图位置、普通父级和 Decorator 所有者; BehaviorTreeRunner 针对 Actor 与字典黑板执行树; BehaviorTreeComponent 把树、ActorPath、Tick 模式和生命周期暴露为可复用场景组件。编辑器中，BTEditorView 管理 Palette、Toolbar、Graph、Inspector、Schema Editor、历史和 Live Debug，另外两个控件负责画布输入、连线和卡片显示。

4.2 资源模型与校验

普通层级使用 parent_id; Decorator 使用 decorator_parent_id，因此不会成为普通控制流子节点。子节点按 X 坐标排序，X 相同时按 Y 排序，使画面从左到右与执行顺序一致。移动同级卡片可以有意改变优先级，序列化则保存布局。

校验拒绝缺失 Root、重复 ID、无效父级、非法 Decorator 归属和节点专属结构错误。Root 不得有父级，叶节点不得拥有普通子节点，Decorator 必须依附有效普通节点，必需参数也必须存在。保存和测试使用同一校验入口。

黑板 Schema 支持 Bool、Int、Float、String 和 Vector2 类型、默认值和动态键策略。严格模式报告空引用和未声明键。Inspector 同时提供带类型的键选择器和自由输入；反向索引可以列出每个键的引用节点和未使用声明。Schema 修改通过深拷贝节点与条目进入树级撤销重做。

4.3 运行时语义

每次 Tick 返回 SUCCESS、FAILURE 或 RUNNING。节点记忆按 ID 保存，并在重启、换树或分支中断时清理。表4.1概括节点语义。

表 4.1: 已实现的运行时节点语义。

节点	行为
Root	执行唯一普通子节点。
Sequence	按序执行，遇失败或运行停止，并记忆运行索引。
Selector	按序执行，遇成功或运行停止，并记忆运行索引。
Reactive Selector	每 Tick 从高优先级重新检查，可中断低优先级分支。
Random Selector	使用可设种子的随机选择，运行期间保持同一子节点。
Parallel	Tick 多个子节点并应用成功/失败阈值。
Repeat	固定次数或无限重复，跨 Tick 保存计数。
Wait	累积 delta，到达时间后成功。
Condition	调用 Actor 判断或比较黑板值。
Action	调用 Actor 方法，把 Bool 或状态转成树状态。

Action 和 Actor 模式 Condition 统一调用 method(blackboard, delta, node)。缺失方法会安全失败并生成调试原因，而不是触发致命脚本错误。黑板条件支持存在、不存在、真假、相等、不等和数值大小比较。

Decorator 在所属节点前或周围执行，支持黑板比较、Cooldown、Time Limit、Invert、Force Result 和 Repeat。Cooldown 防止立即重入，Time Limit 在超时时使运行子节点失败，Invert 交换成功与失败，强制结果只覆盖终态而保留运行态。子树复位同时清理 Decorator 和子节点记忆。

4.4 编辑器编写工作流

插件注册为底部面板。Palette 支持点击和拖拽创建；节点可以移动、从底部方形端口连接到顶部方形端口、断开、复制和删除；复合节点支持多个子节点；

右键菜单提供常用操作和子树显示命令。所有数据改动都推入树快照，因此撤销重做覆盖创建、删除、移动、连线、参数、Schema 和折叠状态。

Inspector 为常见字段提供类型化控件，不要求手写 JSON。Action 编辑方法名；Condition 编辑 Actor/Blackboard 模式、Key、Operator 与值；Random、Parallel、Repeat、Wait 和 Decorator 暴露对应策略。Advanced JSON 仍作为兼容入口，无效 JSON 不会持续输出红色错误，并在修复前保留旧有效参数。

连线需要特殊处理：Godot 原生 GraphEdit 曲线使用侧面 Slot，而本项目采用上下端口。单连接模式隐藏原生持久层并绘制自定义上下边，拖拽预览期间临时恢复原生层。自定义线段命中保留右键断开和撤销重做；关闭该功能则恢复原生线，形成安全回退。

4.5 LIVE DEBUG 与黑板检查

Runner 把 Actor、树路径、活动节点、有序路径、节点状态、失败原因、黑板值和 Schema 信息写入进程专属临时文件，再原子发布到 `res://.godot/behavior_tree_runtime_debug.json`，避免编辑器读取半个 JSON。若快照截断，编辑器保留上一完整状态并在下一帧恢复。

编辑器按当前树过滤快照，显示路径、叶节点、路径摘要、失败菜单和实时黑板。卡片使用状态边框与标注；非活动分支可淡化到 0.24，活动路径保持 1.0。长路径和当前选择使用独立可滚动行，避免 1600×900 下重叠。游戏中没有相应覆盖 UI。

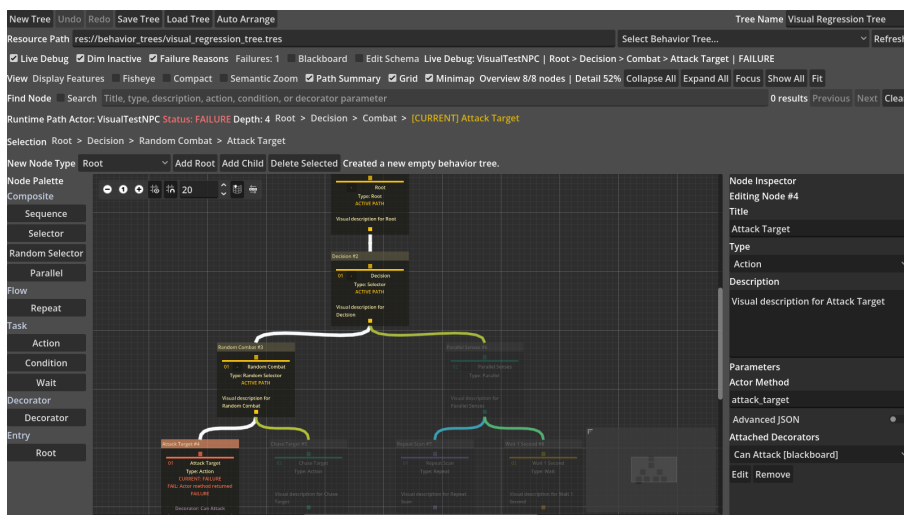


图 4.2: 仅位于编辑器中的活动路径和失败原因显示。

4.6 大型树显示机制

19 项功能通过统一开关系统登记，状态保存在 Godot ConfigFile。组合测试会启用功能、重建、全部关闭，并检查残留。

4.6.1 信息密度控制

Compact 减少卡片宽高，保留类型编码和短标题；Semantic Zoom 不改变几何，而是按缩放层级决定字段可见性；形状/图标补充颜色，可选色盲安全配色提供冗余编码。

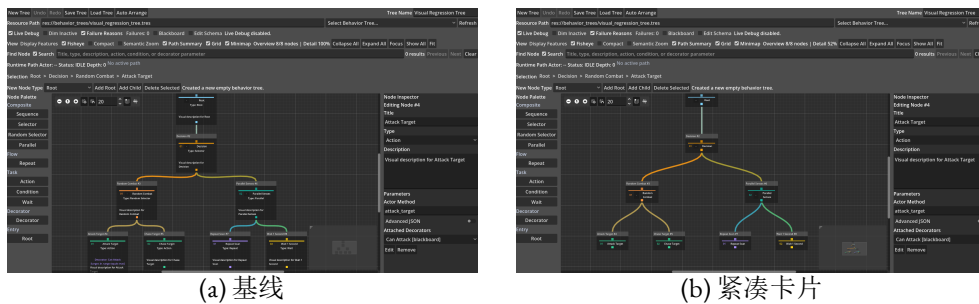


图 4.3: 同一行为树的完整细节基线和紧凑显示。

4.6.2 焦点与结构裁剪

Fisheye 最多把鼠标所在及附近卡片放大 1.20 倍，保留远处上下文。实现不再修改 `GraphNode.scale`，而是改变内部尺寸与字体，并围绕中心补偿；关闭时恢复所有值。Collapse 隐藏后代并显示数量与下两层摘要，Focus 隔离所选分支；Stable Layout 尽量复用无冲突位置。

4.6.3 导航与概览

Search 匹配标题、类型、描述、方法和 Decorator 参数，并用 F3/Shift+F3 导航；Breadcrumb 显示祖先；Fit 计算完整树缩放，大树最低支持 0.10；230×150 小地图表示全部节点和当前视口；Multi-column 平衡宽分支，并用顺序提示保持执行优先级可见。

4.6.4 连线与运行表示

Orthogonal 使用直角边，Bundling 让同方向连线共享主干。Active Path、Branch Dimming、Decorator Badge、Path Summary 和 Failure Annotation 提供运行解释，但不修改行为树资源。它们保持可选，因为不同密度和任务对高亮与连线的偏好不同。

4.7 运行时拓扑缓存

无缓存时，Runner 会反复线性扫描数组以按 ID 找节点、查父级子节点和查所属 Decorator。可选缓存建立 ID 到节点、父 ID 到有序子节点、Owner ID 到有序 Decorator 三种索引，Condition、Action 和黑板动态结果从不缓存。

只比较资源实例和节点数不安全，因为树可以在节点数不变时原地修改。最终实现每个外部 Tick 计算一次精确签名，包括 Root ID、节点数、每个节点资源实例、节点 ID、父级、Decorator Owner 和 X/Y 排序位置。变化后重建索引，递归执行在该 Tick 内复用已验证缓存。这以一次 $O(n)$ 扫描换取正确性。

4.8 演示游戏

复杂二维竞技场提供合成树之外的行为证据。玩家可以移动、近战、冲刺、治疗和短暂隐身；三个守卫运行行为树组件。低血量时守卫撤退并治疗；否则左右攻击分支检查范围、方向、冷却和时限；追击分支接近可见玩家；玩家隐身时 Search 前往最后位置；巡逻使用 Random Selector、Parallel、Repeat 和 Wait。检测采用滞回：330 像素内获得可见目标，超过 460 像素才释放，避免边界抖动。游戏使用简单生成图形和贴图切换，把重点保留在决策逻辑并提高自动测试稳定性。

5 结果与分析

5.1 功能正确性

最终核心回归包含 447 条断言，全部通过（表5.1）。180 物理帧竞技场烟雾测试保持三个活动 Runner 和一个存活复杂敌人。活动项目、分发模板和洁净安装项目日志均没有真实失败、脚本错误、泄漏、崩溃或非法内存访问。

表 5.1: 最终核心自动回归。

套件	通过	总数	主要覆盖
资源与运行时	153	153	结构、节点语义、Decorator、Actor、Schema、调试桥和缓存失效
编辑器 GUI	185	185	图编辑、历史、类型控件、Schema、实时黑板和 19 项开关
基础游戏	13	13	三个敌人、移动、伤害、巡逻、死亡重启与生命周期
复杂竞技场	26	26	索敌、追击、方向攻击、搜索、撤退、恢复和新复合节点
真实 GPU 视觉	70	70	17 张 1600×900 截图和视觉不变量
合计	447	447	核心断言全部通过

附加套件单独报告：人体实验固定场景 GPU 测试 21/21，运行时性能和拓扑安全 511/511，发布包 53/53。0.9.0 ZIP 只包含插件目录、MIT 许可证和 SHA-256 清单，并能在空 Godot 4.6 项目启用。

复杂场景证明敌人由树而非平行硬编码控制器选择行为：进入检测范围写入黑板并选择 Chase，进入对应方向攻击范围选择 Attack，隐身导致丢失目标和最后位置搜索，低血量选择 Retreat 与 Heal。Actor 方法负责实际移动和伤害，但分支选择、顺序与中断来自资源。

5.2 显示密度与上下文结果

表5.2列出主要结果。Compact 在保留全部节点时将卡片总面积稳定降低 55.9%；Semantic Zoom 在低细节把代表字段从六项降到一项，即 83.3%，但不改变几何。

这些结果证明密度变化，不代表任务成绩。

表 5.2: 固定生成树的主要显示测量。

指标	31 节点	121 节点	364 节点
基线卡片面积 (px ²)	1,162,500	4,537,500	13,650,000
紧凑卡片面积 (px ²)	512,864	2,001,824	6,022,016
紧凑面积降低	55.9%	55.9%	55.9%
Focus 可见节点	7	8	9
Focus 可见节点降低	77.4%	93.4%	97.5%
Collapse 可见节点	2	2	2
Collapse 可见节点降低	93.5%	98.3%	99.5%
Search 淡化节点	30	120	363
鱼眼目标增益	1.20x	1.20x	1.20x
小地图覆盖	100%	100%	100%

Focus 随树变大而更具选择性:分别显示 7/31、8/121 和 9/364 个节点;Collapse 更强，三个场景都只保留两个入口，在大树上分别降低 97.5% 与 99.5%。Search 保留布局，只淡化目标外节点，大树为 99.7%。小地图在所有规模都表示全部节点。

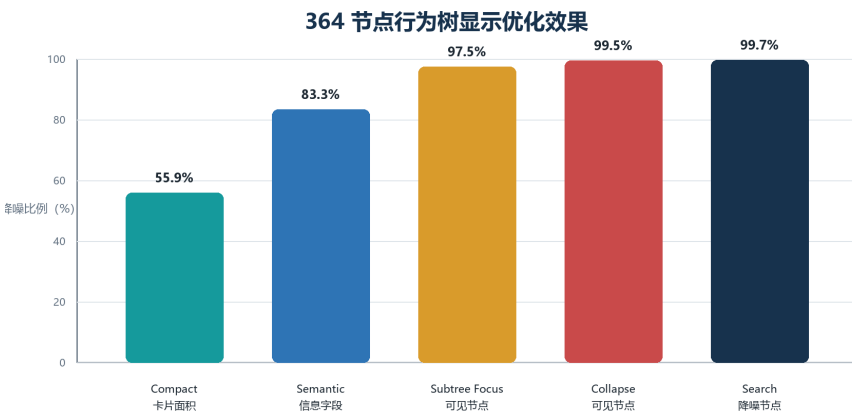


图 5.1: 364 节点树上主要方法的几何和显著性效果。

鱼眼结果有意保持较小：目标放大 20%，在不引起强烈位移的前提下增强局部。它不减少节点数，因此在该指标上不能与 Collapse 直接比较。

5.3 路径与概览结果

非活动分支淡化使用活动 alpha 1.0、非活动 0.24，即 4.17:1 对比；三种树淡化比例为 83.9%、97.5%、99.2%。首次应用从 1.902 ms 增长到 45.037 ms，稳态保持在

0.39662–0.43618 ms。说明主要成本是首次向大量卡片应用状态，而无变化更新较低。

表 5.3: 活动分支淡化与增强小地图。

节点	淡化	首次淡化 (ms)	稳态淡化 (ms)	地图覆盖	地图稳态 (ms)
31	26	1.902	0.40548	100%	0.00132
121	118	6.256	0.43618	100%	0.00140
364	361	45.037	0.39662	100%	0.00174

小地图固定为 230×150，分别表示 31、121、364 个节点。开关周期从 9.3163 ms 增长到 126.79382 ms，稳态状态更新低于 0.002 ms，区分了构建成本和持续观察成本。

5.4 视觉回归发现

视觉测试发现了数字指标无法充分表示的问题。早期鱼眼直接 Transform GraphNode，导致滚轮缩放时全部节点先缩小后放大，关闭后残留 Scale，焦点还偏到鼠标右侧。改为内部尺寸与字体、中心补偿和显式复位后，三个问题全部消失。

Search 曾在 Inspector 中选择远处卡片，却因图坐标和缩放 ScrollOffset 混用而没有居中。按照 GraphEdit.zoom 换算后修复，并加入远距目标必须进入视口的回归。按每层实际最大高度布局也消除了带多个 Decorator 的 36 节点树重叠。

其他视觉结果包括：每个关系只有一条持久线；上下方形端口清楚；运行路径与 Selection 行不重叠；复位后没有残留淡化；Decorator 摘要、Schema Key 和实时黑板类型正确。

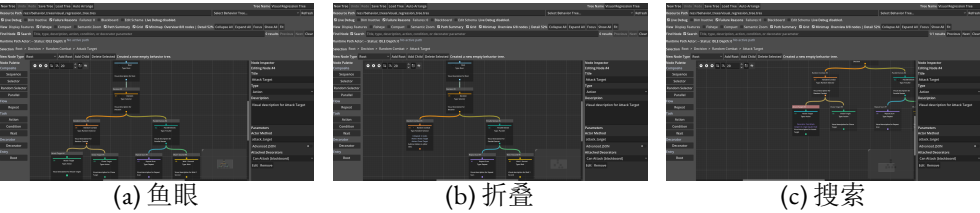


图 5.2: 焦点 + 上下文、结构裁剪和显著性导航的实现示例。

5.5 运行时缓存结果

所有缓存和非缓存实验都到达相同 success 状态。安全测试还在节点数不变时分别修改同级顺序、父级、Decorator Owner 和同 ID 资源实例，均能触发失效；整树替换和关闭缓存回退也通过。

表5.4为七次样本的 Tick 中位值。所有条件下安全缓存都降低成本，且收益随树规模显著增加。31 节点降低 15.98–26.46%，121 节点降低 52.61–56.02%，364 节点降低 76.65–78.00%。

表 5.4: 按树规模和 Runner 数统计的运行中位值。

节点	Runner	无缓存 ($\mu\text{s}/\text{Tick}$)	缓存 ($\mu\text{s}/\text{Tick}$)	降低
31	1	314.834	264.522	15.98%
31	10	376.313	276.741	26.46%
31	50	372.556	294.096	21.06%
121	1	2405.729	1140.190	52.61%
121	10	2429.358	1068.500	56.02%
121	50	2687.255	1270.075	52.74%
364	1	15963.829	3727.207	76.65%
364	10	15926.812	3635.463	77.17%
364	50	15318.540	3370.560	78.00%

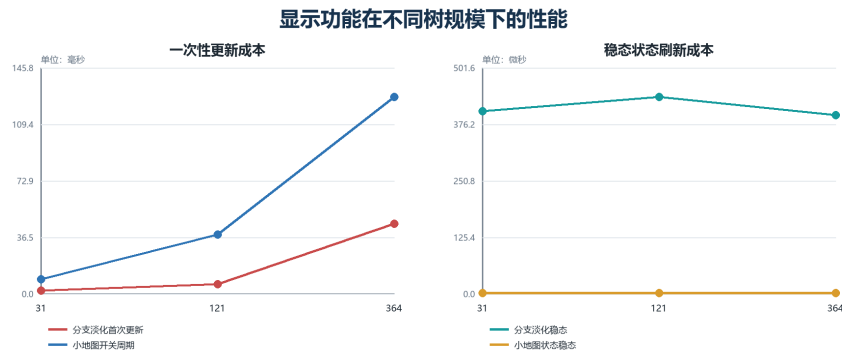


图 5.3: 开启与关闭安全拓扑缓存时的 Tick 扩展。

Runner 数增加时没有清晰单调关系，因为测试固定的是近似总 Tick 工作量，而不是让每个 Actor 在固定墙钟时间内持续运行。有效解释应是同一规模/负载内的缓存与非缓存配对，而不是跨机器生产帧率预测。

5.6 对应研究问题的结果

RQ1 由完整测试矩阵和复杂游戏支持：插件能够编写、序列化、校验、执行和调试非简单智能体。RQ2 在仪器化显示层面得到回答：紧凑、语义、结构、显著性和概览方法产生不同可测效果，并通过真实渲染器复位测试；人的可读性尚未回答。RQ3 在当前工作负载下得到支持：精确签名缓存保持语义，并显著降低大树 Tick 中位成本。

6.1 完整系统作为研究工具

RQ1 关注系统可行性。完成的插件表明 Godot 通用图编辑和扩展 API 足以构成接近成熟工具功能基线的行为树工作流：可序列化资源、可复用 Actor 组件、有序复合节点、条件、动作、Decorator、黑板和实时执行检查。复杂竞技场是关键证据，因为只会正确画图的系统仍可能与智能体行为无关；当前检测、追击、方向攻击、搜索和恢复由资源在运行时选择，而 Actor 方法只提供物理效果。

功能广度也说明行为树编辑器不只是普通树查看器：移动同级节点会改变优先级，隐藏 Decorator 会遮蔽失败原因，切换树必须清除执行记忆，图在被检查时仍持续更新。资源、Runner 和编辑器分层降低了耦合，但集成面仍很大。因此 447 条核心断言应被视为对指定场景的证据，而不是不存在任何缺陷的证明。

不实现 Service 节点是合理范围决策。Parallel、Random、Repeat、Wait、类型化黑板和 Decorator 已能支撑复杂演示。增加另一种仿 UE 抽象会扩大运行时和编辑器组合，却不能直接回答主要研究问题。毕业项目应优先保持可辩护、经过测试的边界。

6.2 显示结果的解释

不同显示技术面向不同任务，其百分比不是统一效用分数。Collapse 的 99.5% 可见节点降低适合已知哪些子树可抽象的情况，却会移除拓扑并可能隐藏意外运行分支；Fisheye 的 20% 数值较小，但保留完整结构，可能更适合导航中局部阅读；Search 的 99.7% 淡化适合已知文字目标，却无法帮助不知道名称的用户；Minimap 的 100% 覆盖提供方向感而不是细节。

这种任务依赖性与文献一致。焦点 + 上下文平衡局部和全局 (Furnas, 1986; Lamping et al., 1995)，概览 + 细节提供稳定全局参考但会分散注意 (Cockburn et al., 2008)；多列布局的优势针对大扇出浏览 (Song et al., 2010)，不能推出所有行为树都应重排；Quilts 的路径任务结果 (Bae and Watson, 2011) 支持补充路径摘要，而不是完全替换可编辑节点图。

因此最重要的设计结果不是找到唯一“最佳方法”，而是受控组合。设计

者可以用 Compact 增加密度，用 Minimap 保持概览，用 Search 定位，再用语义细节检查；运行时则用 Branch Dimming 与文字路径。独立开关也便于隔离故障，例如鱼眼残留 Transform 时可以单独关闭而不影响折叠和实时调试。

心理地图只得到部分处理。Stable Layout 和中心补偿鱼眼减少无关移动，折叠保留 Owner 与摘要；但新子树与已有位置冲突时，自动布局仍可能移动节点。相关研究强调空间连续性 (Dogrusoz et al., 2018; Misue et al., 1995)。未来应测量用户多次折叠/展开后重访节点的能力，而不只统计重叠。

6.3 LIVE DEBUG 作为可读性机制

运行检查把大树问题从一般拓扑阅读变成时序解释。Path Summary、Active Edge、Inactive Dimming、Failure Annotation 和 Live Blackboard 提供冗余证据：颜色表达位置，文字表达顺序，黑板表达决策输入。对于 Decorator 尤其重要，因为失败可以附在 Owner 上，而不是从从未执行的子节点猜测。

原子快照是编辑器和游戏分进程运行下的工程折中。它简单且可检查，但文件轮询不适合高频历史或远程目标，也只保留最新合并状态。Socket 或 Godot Debugger 协议可支持事件时间线和远程 Actor，但同步复杂度更高；对当前项目而言，稳健的最新状态比完整历史更合理。

6.4 缓存权衡

性能实验在固定工作负载下肯定回答 RQ3。索引查询替代递归执行中的重复线性扫描，因此树越大收益越高。精确签名避免了常见错误：把资源身份和节点数相同误认为拓扑未变；测试证明节点数不变的结构修改也能正确失效。

缓存并非免费。每次外部 Tick 都有 $O(n)$ 签名扫描，所以 31 节点收益只有 15.98–26.46%，明显低于 364 节点。事件驱动版本号可以达到 $O(1)$ 校验，但前提是所有修改入口都可靠递增；Godot Resource 可以直接编辑，仅靠版本号会降低安全性。未来可在编辑器变更信号和周期性精确校验之间采用混合方案。

基准还包含 GDScript 和测试框架开销，且只运行在一台 Windows 笔记本。Runner 数条件固定总工作量，因此不是 50 个生产智能体持续 Tick 的精确帧成本。结果只证明控制配对下的相对效果。

6.5 可靠性与发布

严格日志策略发现了断言可能漏掉的问题，包括异步 Timer 在场景退出后存活，以及 PowerShell 原生管道触发 Godot Console 退出期崩溃。改用节点本地状态机消除生命周期泄漏，直接启动 Godot 并指定绝对日志路径避开管道问题。调试插件本身不能制造误导错误，因此这些修复具有实际意义。

洁净安装测试验证 ZIP 边界、清单哈希和空项目启用，而不只是检查文件存在。剩余发布工作包括最终图标、Asset Library 描述和明确目标版本后的兼容矩阵。

6.6 有效性威胁

6.6.1 构念有效性

卡片面积、可见比例、字段数和淡化数测量的是表示属性，不是理解。更小的视图可能隐藏任务所需信息，更强的淡化可能降低对备选项的察觉。因此不能把 55.9% 面积降低写成 55.9% 可读性提高，必须通过参与者实验验证。

6.6.2 内部有效性

生成树结构一致有利于比较，却可能比不规则人工树更容易搜索和布局。时间受操作系统调度和预热影响，七次样本与中位数只能减少而不能消除噪声。固定截图也不能覆盖所有开关组合。

6.6.3 外部有效性

测试限于 Godot 4.6、Windows 和一台 NVIDIA GPU。Godot 内部 GraphEdit 层可能跨版本改变，尤其单线模式会隐藏原生子层。结果不能直接推广到移动编辑器、不同 DPI 或数千节点树；二维战斗 AI 也不能代表机器人等安全要求更高的用途。

6.6.4 研究者与参与者有效性

开发者同时设计系统和自动指标，可能使测试偏向已有机制。公开原始数据和边界有助于审计，但仍需要独立参与者。当前没有参与者数据，因此没有夸大小样本的风险，同时也没有人类可用性结论的依据。

6.7 实践启示

小树适合完整细节节点图；中大型树应按任务提供工具，而不是强制单一自适应模式：Compact/Semantic 用于概览，Collapse/Focus 用于子系统编辑，Search 用于已知目标，Minimap 用于方向，Active Path/Failure 用于诊断。稳定几何应作为安全默认，强畸变功能应可选且易于复位。

对插件开发而言，显示创新必须和编写语义一起测试：连线即使好看，不能点击断开就不完整；折叠若允许新节点无摘要地隐藏就不完整；Live Debug 若因半条消息清空正确状态就不安全。因此集成测试本身也是贡献的一部分。

7 结论

7.1 总结

本文研究如何在 Godot 4.6 中构建完整可视化行为树，同时使大型树更容易检查。项目面对两个相互连接的问题：Godot 提供通用图编辑接口，却没有完整行为树流程；传统完整细节节点图会随着节点、参数和运行标注增加而拥挤。

最终 0.9.0 插件实现可序列化树、类型化黑板、Decorator、有序与响应式复合节点、随机与并行控制、Repeat/Wait 记忆、Actor 方法 Action/Condition、可复用 NPC 组件、撤销重做和仅编辑器 Live Debug。复杂二维场景展示索敌、追击、方向攻击、最后位置搜索、撤退和治疗。

RQ1 在定义范围内得到肯定回答：Godot 编辑器与 Resource API 可以支撑实用的编写、执行和调试流程。RQ2 在仪器化显示层面得到回答：Compact 将卡片面积降低 55.9%，Semantic 将代表字段降低 83.3%，Focus/Collapse 最多降低 97.5%/99.5% 可见节点，Search 淡化 99.7% 非目标节点，Minimap 保持 100% 覆盖；真实 GPU 回归确认这些效果能够安全开关，并消除了鱼眼、居中、重叠和双线问题。RQ3 在当前工作负载下也得到肯定回答：精确签名缓存保持执行结果，并将 Tick 中位成本降低 15.98–78.00%，大树收益更高。

7.2 贡献

本项目贡献了可用 Godot 插件，而不是脱离运行时的视觉原型；贡献了集成的可切换大树与运行检查机制、安全拓扑缓存，以及包含核心 447/447、固定视觉 21/21、性能 511/511、发布包 53/53、原始测量和固定截图的可复现证据包。它也提供一个重要的方法论结论：几何压缩不能直接报告为人的可读性已经改善。

7.3 局限

当前证据限于一台 Windows/GPU、Godot 4.6 和最多 364 节点的确定性树。自动结果能证明实现属性，却不能证明长期编写效率或认知负荷。调试桥提供最新状态而非完整时间线。Service、可复用子树资源、Asset Library 和跨版本矩阵不在已完成范围内。

7.4 未来工作

最优先工作是按已准备协议招募 8–15 人, 对 Baseline、Compact、Collapse、Fisheye、Search、Minimap 进行节点定位、路径解释和 Decorator 编辑实验, 记录时间、错误、平移缩放、成功率和主观评分, 且不能把不可直接比较的几何条件合并成无依据的总分。

工程上可增加可复用子树接口、暂停/单步调试时间线、多 Runner Actor 选择, 以及“事件版本号 + 周期精确校验”的混合缓存。显示研究可以加入增量动画与可配置兴趣度函数, 并测量重访和心理地图。最后还应验证选定 Godot 版本并准备 Asset Library 上架。

7.5 最终结论

本项目说明毕业设计可以超越对既有编辑器的简单模仿。通过结合可执行语义、仅编辑器解释、可切换焦点与复杂度控制、严格回归和明确证据边界, 该插件既是实用 AI 编写工具, 也是后续研究可读性可视化编程的平台。

参考文献

- Colledanchise, M. and Ögren, P. (2018). *Behavior Trees in Robotics and AI: An Introduction*. CRC Press. doi:10.1201/9780429489105.
- Furnas, G. W. (1986). Generalized fisheye views. *Proceedings of CHI '86*, 16–23. doi:10.1145/22339.22342.
- Lamping, J., Rao, R. and Pirolli, P. (1995). A focus+context technique based on hyperbolic geometry for visualizing large hierarchies. *Proceedings of CHI '95*, 401–408. doi:10.1145/223904.223956.
- Cockburn, A., Karlson, A. and Bederson, B. B. (2008). A review of overview+detail, zooming, and focus+context interfaces. *ACM Computing Surveys*, 41(1), 1–31. doi:10.1145/1456650.1456652.
- Song, H., Kim, B., Lee, B. and Seo, J. (2010). A comparative evaluation on tree visualization methods for hierarchical structures with large fan-outs. *Proceedings of CHI 2010*, 223–232. doi:10.1145/1753326.1753359.
- Bae, J. and Watson, B. (2011). Developing and evaluating Quilts for the depiction of large layered graphs. *IEEE TVCG*, 17(12), 2268–2277. doi:10.1109/TVCG.2011.187.
- Dogrusoz, U. et al. (2018). Efficient methods and readily customizable libraries for managing complexity of large networks. *PLOS ONE*, 13(5), e0197238. doi:10.1371/journal.pone.0197238.
- Misue, K. et al. (1995). Layout adjustment and the mental map. *Journal of Visual Languages and Computing*, 6(2), 183–210. doi:10.1006/jvlc.1995.1010.
- Reingold, E. M. and Tilford, J. S. (1981). Tidier drawings of trees. *IEEE Transactions on Software Engineering*, SE-7(2), 223–228.
- Plaisant, C. et al. (2002). SpaceTree. *Proceedings of IEEE InfoVis 2002*, 57–64.
- Shneiderman, B. (1996). The eyes have it. *Proceedings of IEEE Visual Languages 1996*, 336–343.

Godot Engine contributors (2026a). GraphEdit class reference. https://docs.godotengine.org/en/stable/classes/class_graphedit.html.

Godot Engine contributors (2026b). EditorPlugin class reference. https://docs.godotengine.org/en/stable/classes/class_editorplugin.html.

Epic Games (2026). Behavior Tree in Unreal Engine: Overview. <https://dev.epicgames.com/documentation/en-us/unreal-engine/behavior-tree-in-unreal-engine---overview>.

功能参考

表 A.1: 最终编辑器的独立显示功能。

功能	默认	作用
Fisheye / Focus+Context	开	鼠标邻域最多放大 1.20x。
Subtree Collapse / Expand	开	隐藏后代并显示数量与两层摘要。
Compact Mode	关	仅保留类型编码和短标题。
Active Path Highlight	开	强调当前运行路径。
Non-active Branch Dimming	关	Live Debug 时降低非活动分支 Alpha。
Multi-column Layout	关	平衡宽同级分支并保持顺序提示。
Enhanced Minimap	开	230×150 概览和视口标记。
Semantic Zoom	关	不改变图几何地切换信息细节。
Path Summary View	开	显示 Actor、状态、深度和有序路径。
Decorator Condition Badges	开	在 Owner 卡片总结条件。
Search + Highlight	开	搜索节点/Decorator 并淡化不匹配项。
Orthogonal Edges	关	使用直角连线。
Edge Bundling	关	密集区域共享主干。
Stable Incremental Layout	关	布局时复用无冲突位置。
Breadcrumb Navigation	开	显示并导航祖先。
Failure Reason Annotation	开	显示节点失败标签与摘要。
Shape / Icon Type Encoding	开	使用形状和图标补充颜色。
Accessibility Palette	关	色盲安全配色与键盘导航。
Single Connection Rendering	开	显示一条可命中的自定义持久线。

复现与证据索引

以下路径均相对于仓库根目录：

活动项目： `testgame/testgame/`

分发插件： `visual_scripting/addons/behavior_tree_editor/`

复杂树： `testgame/testgame/behavior_trees/complex_guard_validation_tree.tres`

测试脚本： `testgame/testgame/tests/`

显示数据： `testgame/testgame/test_results/display_optimization_raw.csv`

性能数据： `testgame/testgame/test_results/runtime_profile.csv`

视觉截图： `testgame/testgame/test_results/visual/`

人体实验协议： `research/display_optimization/human_comparison_study_protocol.md`

发布包： `dist/behavior-tree-editor-0.9.0.zip`

真实视觉回归必须使用 `-rendering-method gl_compatibility`，不能使用 `Dummy Headless`，因为 `Dummy` 无法提供所需 `SubViewport` 纹理。

计划中的参与者实验

协议包含六种显示条件，每种三个任务。参与者先完成统一练习，再进行 18 个平衡顺序试验。软件记录条件、树规模、目标、时间、误选、Zoom、Pan 和成功率；每种条件后给出七点可读性与认知负荷评分。工作簿支持 15 人、270 行。当前没有参与者数据。使用前必须获得所需伦理许可，与导师确认流程，并保证同意和退出程序符合学校要求。

初稿完成清单

正式提交前，作者应：替换身份与导师占位符；按学校政策修改声明并如实说明 AI 协助；在 Tabula 核实截止时间和词数规则；完成或正式移除人体实验并同步更新摘要、结果、讨论和结论；请导师审阅文献、研究问题和证据边界；按院系要求执行 Turnitin；用模板编译并逐页检查；加入最终词数和要求的仓库/归档信息。